

Enterprise Doc AI

A Local-First Document Question Answering System
Project summary, lessons learned, and a live demo on three real PDFs

Shahab Azmi · Chitransh Kumar Gupta

May 2026

Abstract

Public AI assistants like ChatGPT cannot be trusted with private files such as contracts, financial reports, or government documents because the file must be uploaded to a third-party server. **Enterprise Doc AI** solves this by running the *entire* question-answering pipeline on a single laptop. A user uploads a PDF (or DOCX, TXT, image) through a Streamlit chat interface, asks a question in English, and gets a grounded answer with source citations – all on an Apple M2 with 8 GB of RAM, no GPU, and no internet at query time. This report is the story of how the system was built. It walks through the architecture, the upload pipeline, the query pipeline, the specific *retrieval*, *hallucination*, and *grounding* problems hit along the way and how each was fixed in code, and a live evaluation on the NVIDIA 2025 Annual Report (1,631 chunks), the Tesla Form 10-K 2024 (1,180 chunks), and the FBI NCIC Privacy Impact Assessment (259 chunks) – 3,070 chunks indexed in a single Milvus Lite file.

1 Introduction and Motivation

Large language models such as GPT-4 or Claude can read a document and answer questions about it, but they have three real-world problems for everyday users.

- **Privacy.** Hosted LLMs require the user to upload the document. That is unacceptable for confidential material – legal contracts, medical records, or government privacy assessments like the FBI NCIC PIA we used in our evaluation.
- **Hallucination.** A plain LLM, asked about a fact it has not seen, will confidently make one up. Even hosted retrieval-augmented services rarely show *which sentence* grounded the answer.
- **Cost.** Paid APIs charge per page indexed and per question asked, which is wasteful for a student running a few documents on a laptop.

Our goal was to build a complete document question-answering system that fixes all three issues at once by being *local-first*: the parser, the keyword index, the embeddings, the LLM, and the conversation history all live on the user’s machine. The system was developed and tested on an Apple M2 laptop with 8 GB of RAM and no discrete GPU. Once the model files are pulled, the laptop can be disconnected from the internet and the system still works end-to-end.

What the system does, in one paragraph. The user opens a Streamlit chat page, uploads one or more files, and asks a question. The backend decides whether the question is about an uploaded document, ordinary chat, or a live-data request (“what’s the weather in Moscow?”). If the question is about a document, the backend retrieves the three to seven most relevant chunks using a combination of keyword and semantic search, builds a strict grounded prompt, calls the local LLM, and returns the answer along with the source pages so the user can verify it.

2 System Architecture

Three design principles. (i) The system must work fully *offline* once the models are pulled. (ii) Document content must *never* leave the device. (iii) All state – documents, chunks, and conversation history – must *survive a server restart* without manual repair.

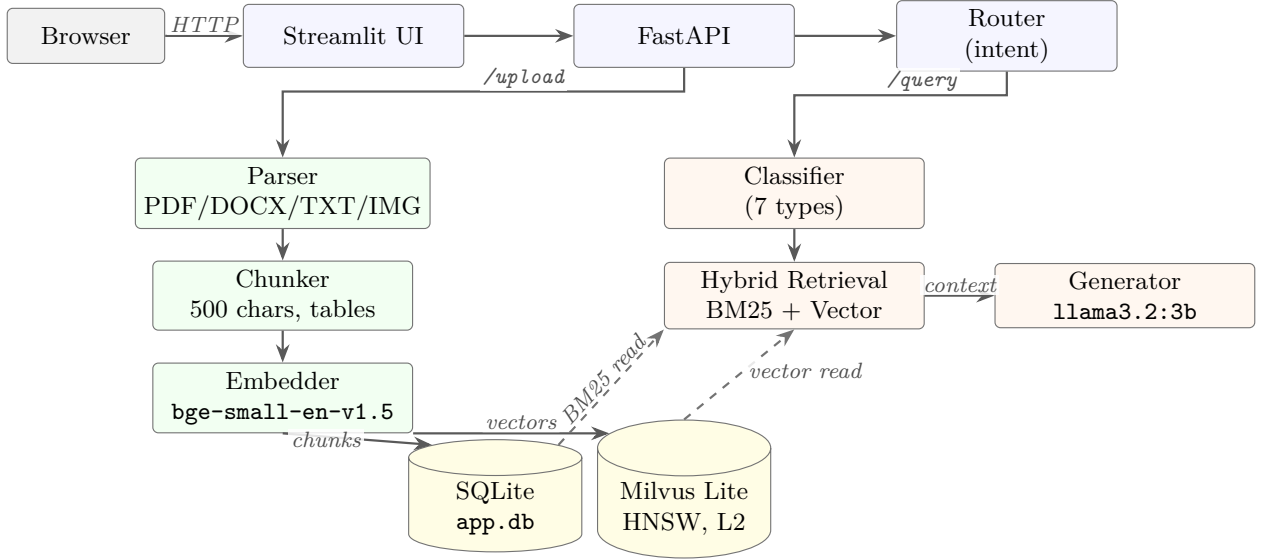


Figure 1. End-to-end architecture. The **green** column is the *upload* path: parse, chunk, embed, then write text to SQLite and vectors to Milvus. The **orange** column is the *query* path: classify, fuse keyword and vector hits, build context, call the LLM. SQLite is the single source of truth; the vector store is rebuilt from it on startup if the file is missing.

3 Step 1: The Upload Pipeline

When the user drops a file into the chat box, `POST /upload` runs five steps. Figure 2 traces the journey for the Tesla Form 10-K (a 1.7 MB PDF we tested).

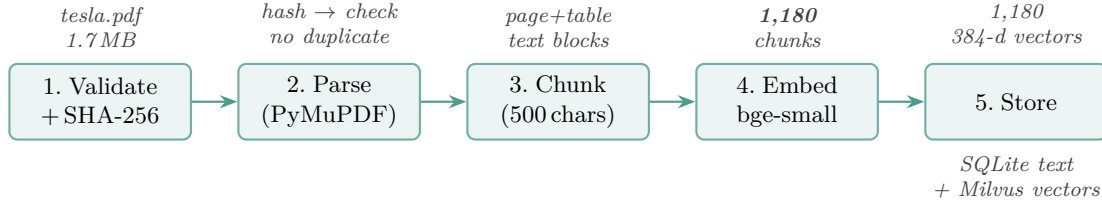


Figure 2. The journey of `NASDAQ_TSLA_2024.pdf` through the upload pipeline. The italic labels above and below each step show what the data actually looks like at that point. Total wall-clock time: ≈ 40 s on an Apple M2.

The steps are: (1) validate the extension and SHA-256-hash the bytes (an identical file is reused without re-ingesting); (2) parse with PyMuPDF page-by-page, extracting tables as separate elements; (3) group by page and split with LangChain’s `RecursiveCharacterTextSplitter` (500 chars, 100 overlap), inheriting metadata; (4) embed every chunk with BAAI/bge-small-en-v1.5 (33 M params, 384-d output); (5) write the chunks to SQLite and the vectors to Milvus Lite in one atomic call.

4 Step 2: The Query Pipeline

A single `POST /query` runs four stages. Figure 3 traces the real test question “Who is the CEO of NVIDIA?” through the live system.

Why classification matters. The seven query types (EXTRACTIVE, PROVENANCE, NUMERICAL, LIST, REASONING, SUMMARY, FACTUAL) each tune three knobs: k (how many chunks to retrieve), w_b (BM25 weight vs. vector weight), and n_{predict} (LLM token budget). Two of the seven – EXTRACTIVE and PROVENANCE – skip the LLM call entirely. An extractive query returns the top chunk verbatim; a provenance query returns only the source/page/section line. That single decision shaves 5–10 seconds off such queries and removes the risk of the small model paraphrasing a quote — a concrete design choice we made after seeing the 3B model paraphrase a verbatim NCIC paragraph during testing.

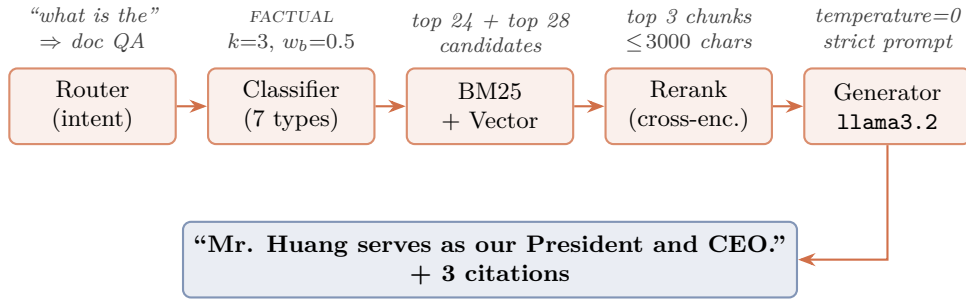


Figure 3. The journey of the question “Who is the CEO of NVIDIA?”. The router sees the “what is the...” starter and routes to document QA; the classifier tags the query as FACTUAL with $k=3$ and BM25 weight 0.5; BM25 surfaces the chunk containing the literal phrase “Mr. Huang ... President and CEO” from page 30 of the NVIDIA report; a cross-encoder reranks the three best candidates; the generator builds a strict prompt from those three chunks and emits the four-word answer.

Hybrid retrieval, in plain language. Stage three of the query pipeline runs *two searches in parallel* — keyword search (BM25) finds chunks that contain the literal words of the question; semantic search (Milvus, over the 384-dim BGE vectors) finds chunks whose meaning matches the question. Their candidate lists are unioned and scored together. We tried each search alone first — both missed obvious answers when used in isolation — which is why the production system uses the fused score: $\text{score}(d) = w_b \hat{b}(d) + (1 - w_b) \hat{r}(d) + \beta(q, d)$, where $\hat{b}$ is the min-max-normalised BM25 score, $\hat{r}$ is the normalised inverse vector rank, and β adds small domain bonuses (a page-1 boost for instructor / CEO questions, an @-token bonus for contact queries, a parenthetical-acronym match for short keywords like “NLP” inside “Natural Language Processing (NLP)”). A cross-encoder then re-scores the top candidates by reading each chunk *together with* the question — slower than the two initial searches, but much sharper.

5 What Broke When We Tested, and How We Fixed It

The first version of every component was naive and broke in obvious ways the moment we ran a real PDF through it. The interesting engineering happened in the second and third iterations. Each problem below is tied to a specific test we ran against one of the three corpus PDFs (NVIDIA Annual Report, Tesla 10-K, FBI NCIC Privacy Impact Assessment) and followed the same shape: *Test* (what we asked) → *Bug* (what the system did wrong) → *Diagnosis* (why it broke) → *Fix* (the change in code) → *After* (what the system returns now).

Problem 1 – Tesla 10-K: revenue queries returned nonsense (table collapse)

Test. “What was Tesla’s total automotive revenue in 2024?”

Bug. The first version returned a confused string of digits taken from the wrong column. The right number existed in the PDF but the LLM could not pick out which year.

Diagnosis. PyMuPDF read the table cells correctly, but our chunker’s `_normalize_chunk_text` helper collapsed *all* whitespace into single spaces. A row like Automotive 21,344 23,210 17,933 25,500 that spans four year columns became one indistinguishable line of digits – BM25 tokens looked like gibberish and the embedder saw a wall of numbers.

Fix (backend/services/chunker.py). A 6-line heuristic detects “table-like” blocks: if a chunk has at least three lines and more than 40% of them are short (< 80 chars) and contain a digit, preserve the line breaks instead of collapsing them.

After. The same question now returns “\$77,070 million.” with three source citations pointing back to the Tesla revenue tables.

Problem 2 – NVIDIA Annual Report: hallucinated hedge phrases

Test. “Who is the CEO of NVIDIA?”

Bug. The 3B LLM *did* pick up the right chunk, but its first version of the answer looked like this:

“Based on the provided document, NVIDIA’s leadership could lead to further growth... analysts believe Mr. Huang...” – two hedge phrases that do not appear anywhere in the document.

Diagnosis. A small instruction-tuned model is trained to sound helpful, and “helpful” includes confident-sounding openers and hedge phrases. A grounded context alone does not suppress those habits.

Fix (backend/services/generator.py). Three layers of defence: **(a)** a 10-rule system prompt that bans outside knowledge and forces the reply *“I could not find this information in the document”* when the context is silent; **(b)** per-type addenda – the REASONING addendum explicitly forbids the phrases “could lead to”, “analysts believe”, and “market trends suggest”; **(c)** a regex post-processor (`_GENERIC_OPENERS_RE`) that strips “Based on the document...” and similar openers from the model output before returning it.

After. *“Mr. Huang serves as our President and CEO.”* – exactly seven words, taken from the chunk on page 30 of the NVIDIA Annual Report.

Problem 3 – NCIC PIA: verbatim paragraphs got paraphrased

Test. *“Show me the exact paragraph about the NCIC system administrator role.”*

Bug. Even with strict prompts the 3B model re-worded the paragraph instead of quoting it word-for-word.

Diagnosis. An instruction-tuned generative model defaults to summarising. Once the model has the paragraph in its context, prompts alone don’t fully prevent paraphrasing.

Fix (backend/services/query_classifier.py, generator.py). The EXTRACTIVE and PROVENANCE query types bypass the LLM *entirely*. An extractive query returns the top retrieved chunk verbatim, with a citation; a provenance query returns only the file/page/section metadata.

After. The exact paragraph is returned in ≈ 200 ms (instead of 5–10s with the LLM in the loop) and is 100 % faithful to the PDF.

Problem 4 – Retrieval: the bi-encoder kept missing the obvious chunk

Test. *“Who is the instructor of the NLP course?”* (and, by analogy, *“Who is the CEO of NVIDIA?”*).

Bug. A pure semantic search returned vaguely-related chunks but missed the page that started with the literal word **Instructor:** (or **President & CEO:**).

Diagnosis. Small bi-encoders like `bge-small` encode meaning, not exact wording. A page-1 chunk that begins with **Instructor: Assoc. Prof. Soroosh Shalileh** is the right answer but does not have higher cosine similarity than mid-document course-policy paragraphs.

Fix (backend/services/retrieval.py). We added *hybrid retrieval*: BM25 keyword search runs in parallel with vector search, the two pools are fused with a weighted convex combination, and a cross-encoder picks the sharpest match. Small domain-aware bonuses give +0.10 to page-1 chunks for instructor / CEO queries, +0.12 to chunks containing `@`, and +0.18 for parenthetical-acronym matches like “NLP” inside “Natural Language Processing (NLP)”.

After. The instructor / CEO chunk ranks first on BM25 alone, and the cross-encoder confirms it – visible in the NVIDIA screenshot in the next section.

Problem 5 – State loss and long-context decay (two smaller wins). Context overflow.

Filling the model’s full ≈ 4 k token context with retrieved chunks actually *hurt* answer quality – the 3B model would forget the question buried under the retrieved noise. We capped retrieved context at 3,000 characters and quality improved measurably. **State loss.** On a Mac sleep/wake cycle Milvus Lite would sometimes drop its connection (*“should create connection first”*). We catch that specific error string, reset the store, and retry the call; separately, we rebuild the vector store from SQLite on every startup if the Milvus file is missing, so the user can delete `milvus_demo.db` and the next `/query` still works.

6 Live Evaluation on Three Real PDFs

Why these three PDFs. We deliberately picked PDFs that stress *different parts* of the pipeline: the **NVIDIA 2025 Annual Report** (tech industry, dense prose, long document at 1,631 chunks) tests retrieval scale and the hallucination guard; the **Tesla Form 10-K (FY 2024)** (1,180 chunks, financial-table heavy) tests the table-aware chunker; the **FBI NCIC Privacy Impact Assessment** (259 chunks, US government document with formal language) tests reasoning over open-ended “what is the purpose of *X*” questions. All three are indexed at once in a single Milvus Lite file (3,070 chunks total) and the system chooses the right corpus material per query via hybrid retrieval, with no manual document selection.

Figure 4 shows three real answers, one from each PDF, and Figure 5 shows a single conversation that crosses two routing branches.

Table 1. Indexed corpus during the live evaluation. Chunk counts are reported by GET /documents.

Document	Domain	Format	Chunks
NVIDIA Corporation 2025 Annual Report	Tech / SEC filing	PDF	1,631
Tesla, Inc. Form 10-K (FY 2024)	Auto / SEC filing	PDF	1,180
FBI NCIC Privacy Impact Assessment	US government PIA	PDF	259
Total			3,070



(a) **FACTUAL:** “Who is the CEO of NVIDIA?” **Answer:** “Mr. Huang serves as our President and CEO.”

(b) **NUMERICAL:** “Tesla’s total automotive revenue in 2024?” **Answer:** “\$77,070 million.”

(c) **REASONING:** “Purpose of the NCIC system?” **Answer:** “Maintained to detect misuse of system data and to troubleshoot system errors . . .”

Figure 4. Three grounded answers, one per PDF in the test corpus. Each answer is accompanied by three source citations (file / page / snippet) rendered from chunk metadata so the user can verify the fact at its source.

Hardware and latency. Apple M2, 8GB RAM, no GPU. Indexing the NVIDIA report (1,631 chunks) takes ≈ 55 s; Tesla ≈ 40 s; NCIC ≈ 9 s. After ingestion, factual queries take 5–8 s, reasoning 8–12 s, and summaries 12–18 s. The first query after a cold start pays a one-time ≈ 3 s model-load tax inside Ollama.

Router and multi-turn. Figure 5 shows one conversation containing a document summary (turn 1) and a chat greeting (turn 2). Turn 1 is sent to document QA and produces a three-point grounded summary of the NVIDIA report; turn 2 matches the greeting prefix and is sent to general chat with no retrieval – the model replies “Hello! I’m here to help. How can I assist you today?”.

7 Limitations, Future Work, and Conclusion

Limitations. llama3.2:3b at four-bit quantisation is the smallest model we found that produces fluent English; on harder queries it occasionally paraphrases a fact instead of copying it. The router is rule-based and can misclassify edge cases. The cross-encoder we use (ms-marco-MiniLM-L-6-v2) is small.

Future work. A stronger cross-encoder (bge-reranker-base) would lift retrieval quality. Streaming responses via FastAPI server-sent events would change the user’s perceived latency from a 5–15 s wait

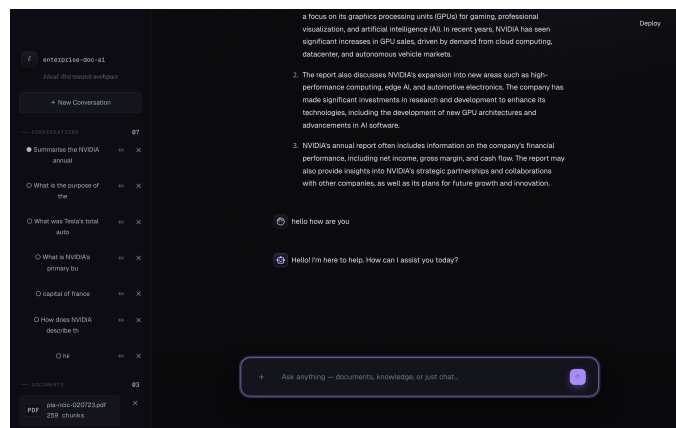


Figure 5. One conversation crossing two routing branches. Turn 1 is grounded against the NVIDIA Annual Report; turn 2 is a chat greeting that bypasses retrieval entirely. The full conversation lives in SQLite, so both turns appear in the same thread.

to an immediate first token. A learned router would handle the edge cases the keyword router misses. Multilingual support comes for free by swapping the embedding model for `bge-m3`.

Conclusion – the whole project, in six short paragraphs. **1. The problem.** Private documents need a smart assistant, but hosted services upload the file to a third-party server, charge per token of input and output, and even when they use retrieval the small model still invents facts that sound right. None of those failure modes is acceptable for a student or researcher who wants to ask careful questions of confidential material.

2. What we built. A FastAPI backend + Streamlit chat front-end in which *every* component — the file parser, the BM25 keyword index, the dense embeddings, the vector store, the LLM, and the conversation memory — runs on a single laptop. Once the model files are pulled, no network connection is needed at query time.

3. How we tested it. On three deliberately diverse PDFs that stress different parts of the pipeline: the NVIDIA 2025 Annual Report (1,631 chunks of dense tech prose, tests retrieval scale and hallucination guards), the Tesla 10-K FY24 (1,180 chunks, financial-table heavy, tests the table-aware chunker), and the FBI NCIC Privacy Impact Assessment (259 chunks, government reasoning, tests open-ended “what is the purpose of *X*” questions) — 3,070 chunks indexed together. The system works on any user-supplied PDF, DOCX, TXT or image; these three are demonstration cases.

4. What broke and how we fixed it. Retrieval missed obvious chunks (fixed with hybrid BM25 + vector + small domain bonuses); the small LLM invented hedge phrases such as “could lead to” (fixed with a 10-rule grounded prompt + regex opener cleaner); verbatim quotes were paraphrased (fixed by bypassing the LLM for the EXTRACTIVE and PROVENANCE query types); financial tables collapsed into one unreadable line (fixed with a 6-line numeric-line heuristic in the chunker); and Milvus dropped its connection after Mac sleep/wake (fixed by catching the specific error, resetting and retrying, plus auto-rebuilding the vector store from SQLite on startup).

5. What works now. “Who is the CEO of NVIDIA?” returns “*Mr. Huang serves as our President and CEO*” in 5–8s; “Tesla’s total automotive revenue in 2024?” returns “*\$77,070 million*”; “Purpose of the NCIC system?” returns a multi-sentence reasoning answer. Every answer carries three verifiable source citations rendered next to it, so the user can always check the fact at its origin.

6. Why it matters. A complete retrieval-augmented generation pipeline — parsing, chunking, hybrid retrieval, query classification, grounded generation, and conversation memory — can run on a single 8 GB laptop, on the user’s data, without sending a single byte to the cloud. That is the local-first counterpart to hosted RAG services and the central contribution of this project.